

06-Python Memory Management and Programming Fundamentals

Introduction

- Covers a Python class session focused on memory management, data types, and input/output operations in Python.
- Emphasis on understanding internal workings of Python programs through both theoretical explanation and practical demonstration using tools like Thonny editor and Python Tutor.

Memory Management in Python

- Python handles memory management automatically (automatic allocation and deallocation).
- Two main aspects: memory allocation (using raw memory allocator, object-specific allocator, Python Virtual Machine—PVM) and memory deallocation (via garbage collector).
- Python users typically do not intervene in memory management; it is handled by the language runtime.
- Memory allocation involves assigning space for objects/data; deallocation is done by garbage collection based on reference counting and periodic cleanups.

Types of Python Objects

- All data in Python is treated as objects (including functions and certain special objects).
- Common object types discussed:
 - **Integer (int):** e.g., 10, 0, -10

- **Float:** e.g., 10.20, -10.20
- **Complex:** e.g., 10+20j, 90j
- **Boolean:** True, False
- **NoneType:** None
- **String:** Enclosed in single, double, or triple quotes
- **List:** e.g., [10, 20, 30]
- **Tuple:** e.g., (10, 20, 30)
- **Dictionary:** e.g., {'a': 10}
- **Set/Frozenset:** e.g., {10, 20}, set is mutable, frozenset is immutable
- **Byte/ByteArray**
- **Range object**
- **Functions, user-defined classes/objects, iterators, generators**
- Each type corresponds to a class internally in Python, with object-specific allocators to manage memory due to their structural differences.

Python Memory Visualization

- The memory allocated for a running Python program is divided into:
 - **Code Area:** Stores the bytecode instructions.

- **Variable Area:** Holds identifiers (names) pointing to data.
- **Heap Area:** The main storage for all objects/data.
- **Stack Area:** Tracks the order of execution (function calls/modules).
- **Free Area:** Buffer space for emergency or additional allocations.
- Execution is visualized step-by-step:
 - Constants/objects stored in the heap; variable names mapped to these objects in the variable area (by memory addresses).
 - The stack tracks active execution contexts (e.g., which module or function is being run).
- Tools:
 - **Thonny Editor:** Step-by-step code execution/showing variable, heap, and stack partitions.
 - **Python Tutor:**
Online tool for animated visualization of code execution, useful for understanding loops, functions, and logic flow.

Input and Output Functions

- **input():** Reads user input as a string by default.
 - Typecasting (using `int()`, `float()`, etc.) is needed to process numerical input.
 - `eval()`
function can evaluate user input as an appropriate Python expression, offering flexibility for input types (int, float, list, etc.).

- **print():** Primary output function in Python.

- Can take multiple arguments (separated by commas), default separator is a space.

- Attributes:

- **sep:** Changes separator between multiple arguments.

- **end:**

Changes what is printed at the end (default '\n'). Can be set to customize line-breaks or add other characters.

- Demonstrated use of escape characters (\n, \t, \v, \f, \\) for formatting output.

String Formatting and Output Customization

- Use of **f-strings** (e.g., `f"The sum {a} and {b} is {c}"`) for concise, readable string formatting with placeholders.
- Enables dynamically inserting variable values into output strings and minimizing code repetition.
- Guidance to minimize code lines in Python by leveraging built-in features.

Practical Tips & Tools

- Encouraged to use code visualization tools (Thonny, Python Tutor) to step through and understand program logic, especially for beginners.
- Debugging stepwise helps build logical understanding, an emphasis for non-programming-background learners.
- Assignments and course notes are available on Google Drive; students are encouraged to read and revisit basic concepts.

Q&A and Additional Notes

- Various student questions on topics like heap vs variable area, memory address assignment, difference between set and frozenset, object-oriented programming in this course, and more were answered.
- Details on how Python's garbage collector works (reference counting, periodic cleanup—automatic and also callable manually, though not generally recommended).
- Advice on Python's efficiency in code length compared to other languages (e.g., Java).
- Programming component topics like identifiers, expressions, comments, blocks/indentation, reserved words were introduced as upcoming content.
- First assignment to be given after covering operators, with notes and further resources available online.

End of summary

Notes powered by [CraftNote](#)